

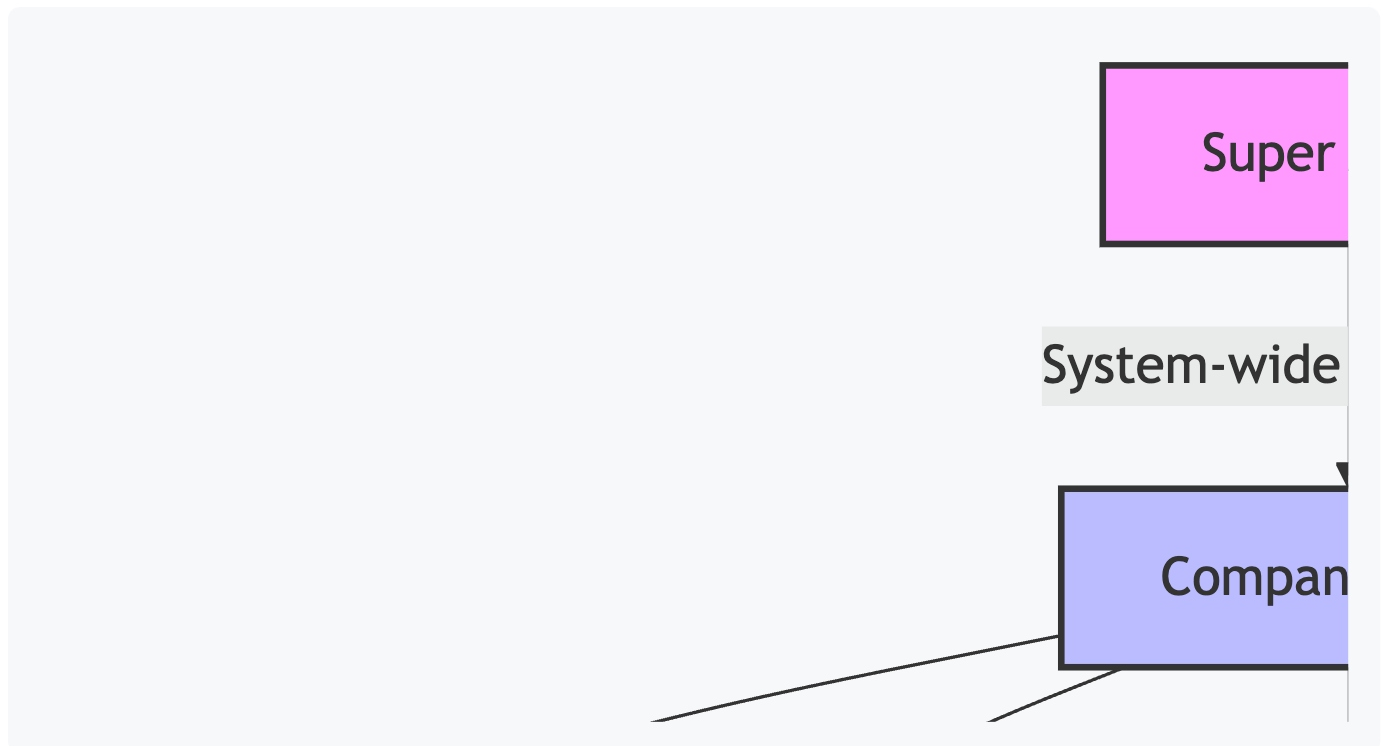
Role-Based Access Control (RBAC) - Software Requirements Specification (SRS)

1. Overview

The LawDocs Compliance platform utilizes a multi-dimensional Access Control system designed for B2B SaaS multi-tenancy. It integrates four core dimensions of access control:

1. **Role-Based Access Control (RBAC):** Hierarchical roles with predefined action permissions.
2. **Multi-Tenant Data Isolation:** Hard segregation of data using `companyId`.
3. **Module-Based Access Control:** Feature-flagging/module toggles per user.
4. **Departmental Scoping:** Scoped administrative powers for specific departments (`sub-admin`).

2. Role Definitions & Hierarchy



2.1 Super Admin (`super-admin`)

- **Scope:** System-wide
- **Description:** Highest level of access. Manages the entire platform, all companies, and all users.
- **Bypasses:** Module-level restrictions and company-level isolation (where applicable).

2.2 Company Admin (`admin`)

- **Scope:** Tenant-wide (Company level)
- **Description:** The primary account owner for a specific company tenant. Their `companyId` is their own `_id`. They can create and manage sub-users (`sub-admin` , `maker` , `checker` , `auditor`) within their company.
- **Bypasses:** Module-level restrictions (has access to all modules available to the company).

2.3 Sub-Admin (`sub-admin`)

- **Scope:** Department-wide (Tenant level)
- **Description:** Inherits admin-like powers but restricted to specific assigned departments.
- **Inheritance:** Automatically passes any authorization checks that allow `admin` , `maker` , `checker` , or `auditor` .

2.4 Maker (`maker`)

- **Scope:** Tenant-level, Module-restricted
- **Description:** The operational user responsible for data entry, compliance execution, and uploading evidence.

2.5 Checker (`checker`)

- **Scope:** Tenant-level, Module-restricted
- **Description:** The reviewing authority who approves or rejects the compliance actions submitted by the Maker.

2.6 Auditor (`auditor`)

- **Scope:** Tenant-level, Read-Only / Audit-focused
- **Description:** Internal or external auditor who views compliance responses and generates audit reports. Cannot modify operational data.

2.7 Viewer (`viewer`)

- **Scope:** Tenant-level, Read-Only
- **Description:** Strictly read-only access to view dashboards and download evidence.

3. Granular Permission Mapping

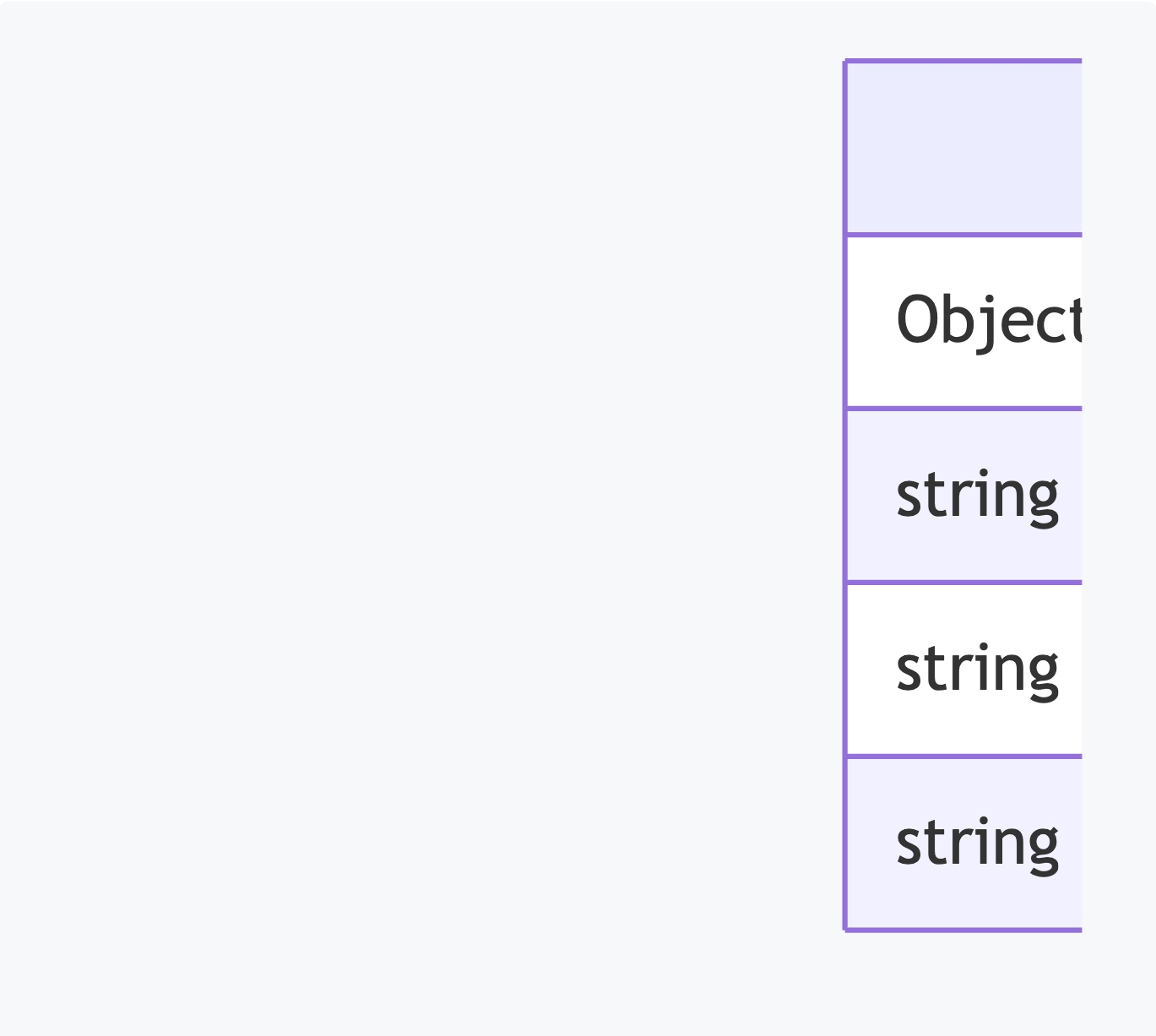
Role	Permissions
	<code>create_response</code> , <code>edit_response</code> , <code>mark_complied</code> , <code>review_response</code> ,

Admin	<code>approve_response</code> , <code>reject_response</code> , <code>view_all_responses</code> , <code>view_all_dashboards</code> , <code>audit_responses</code>
Maker	<code>create_response</code> , <code>edit_response</code> , <code>upload_evidence</code> , <code>mark_complied</code> , <code>view_own_responses</code> , <code>view_maker_dashboard</code>
Checker	<code>view_marked_responses</code> , <code>review_response</code> , <code>approve_response</code> , <code>reject_response</code> , <code>view_checker_dashboard</code> , <code>download_evidence</code>
Auditor	<code>view_all_responses</code> , <code>audit_responses</code> , <code>generate_audit_reports</code>
Viewer	<code>view_responses</code> , <code>download_evidence</code>

4. Multi-Tenant Data Isolation & ER Diagram

Every user object (except potentially super-admin) is strictly tied to a `companyId` .

- **Admins:** `companyId` = `user._id`
- **Sub-Users:** `companyId` = `creator_admin._id` (stored in DB)





Security Rules:

- JWT tokens are validated, but the `companyId` is **always derived from the database** (not the JWT payload) to ensure stale tokens cannot exploit changed company associations.
- The `verifyCompanyAccess` middleware intercepts requests and asserts that `req.user.companyId === req.params.companyId`.

5. Module-Based Access Control

The platform enforces access down to the specific business module level. Users (`maker`, `checker`, `auditor`) possess an `accessibleModules` array. `admin` and `super-admin` bypass this check.

Module Hierarchy

If a user is granted a parent module, they recursively gain access to all child modules.

- **MIS** (`mis`)
 - Task Management (`mis:task_management`)
 - Policy Tracker (`mis:policy_tracker`)
 - Committee Tracker (`mis:committee_tracker`)
 - Asset Management (`mis:asset_management`)
 - HRMS (`mis:hrms`)
 - Nidhi AI (`mis:nidhi_ai`)
- **Liquidity Risk** (`liquidity_risk`)
- **RBI Compliance** (`rbi_compliance`)
- **IT & Data Compliances** (`it_data_compliance`)
- **ROC Compliance** (`roc_compliance`)
- **Labour Law Compliances** (`hr_compliance`)

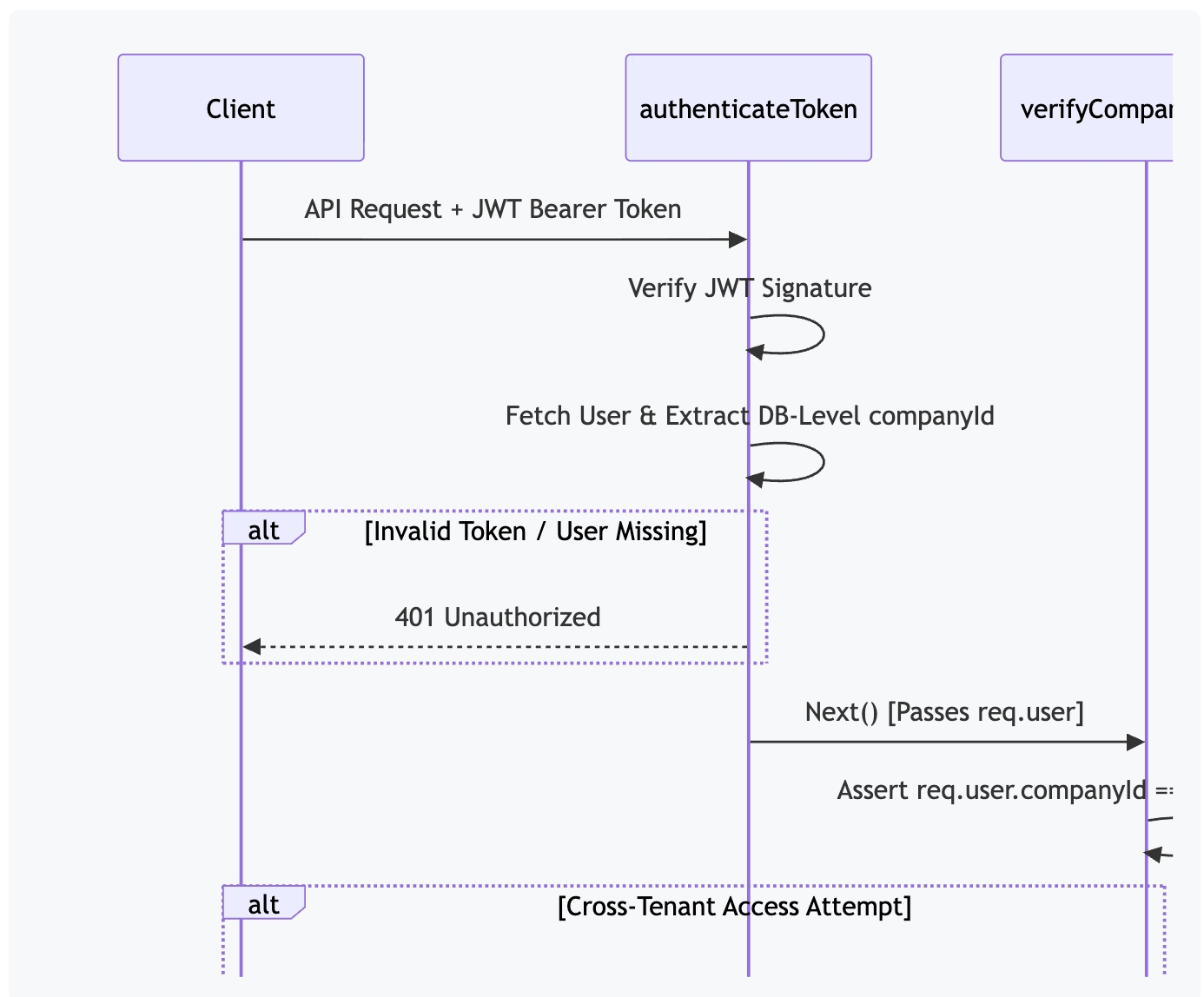
- **Labour Law Compliances** (`in_compliance`)
- **Reports** (`reports`)

6. Authentication & Session Management

- **Token Validation:** Standard Bearer JWT verification.
- **Session Invalidation:** System checks if `passwordChangedAt` timestamp is greater than the JWT `iat` (issued at) timestamp. If so, the session is killed, forcing a re-login.

7. Request Authorization Flow

The diagram below maps how a request traverses the layered middleware security system to ensure absolute safety.



8. Middleware Implementation Guide (Recreation)

To recreate this RBAC on any route, chain the following middlewares:

1. `authenticateToken` : Validates JWT, fetches User from DB, attaches `companyId` , `role` ,

`accessibleModules` , and `departments` to `req.user` .

2. `verifyCompanyAccess` (*Optional*): Ensures cross-tenant data leakage does not occur for targeted API endpoints.
3. `authorizeRoles(['role1', 'role2'])` : Verifies if the user's role is in the allowed list (Note: `sub-admin` implicitly passes if `admin` , `maker` , `checker` , or `auditor` is allowed).
4. `checkPermission('permission_name')` (*Optional*): Granular action check against `ROLE_PERMISSIONS` .
5. `checkModuleAccess('module_key')` (*Optional*): Validates if the user's `accessibleModules` array contains the requested module or its parent.

Example Route Implementation:

```
router.post(
  '/compliance/:companyId/mark-complied',
  authenticateToken,
  verifyCompanyAccess,
  authorizeRole(['maker', 'admin']),
  checkPermission('mark_complied'),
  checkModuleAccess('rbi_compliance'),
  controller.markComplied
);
```